

Paradigmas de Programación

EIF-400

Proyecto Enigma

Estudiantes:

Alisson Cordero Fonseca

Melany Céspedes Sánchez

Universidad Nacional de Costa Rica

Sede Regional Brunca

Campus Pérez Zeledón

Profesor: Pablo Venegas Elizondo

II Ciclo

2026

Tabla de contenidos

1. Introducción 4

2. Planteamiento del problema 4

2.1 Problema o motivación 4

2.2 A quiénes afecta 4

2.3 Contexto 4

2.4 Soluciones existentes y sus limitaciones 4

3. Evidencias de investigación 4

3.1 Análisis de soluciones existentes 4

3.2 Fuentes y evidencia 5

4. Propuesta de solución 5

4.1 Características principales 5

4.2 Beneficios esperados 6

4.3 Ejemplo de programa que puede desarrollarse 6

4.4 Relación entre el problema y el lenguaje 6

5. Objetivos 7

5.1 Objetivo general 7

5.2 Objetivos específicos 7

6. Alcance y limitaciones 7

6.1 Alcance 7

6.2 Limitaciones 7

7. Descripción del lenguaje 8

7.1 Identidad 8

7.2 Ejemplo de código 8

8. Palabras reservadas 9

9. Sintaxis 10

10. Semántica 12

10.1 Delimitador de instrucción 12

10.2 Inicio del programa y bloques 12

10.3 Variables y constantes 12

10.4 Alcance (scope) 12

10.5 Evaluación de expresiones 12

10.6 Condiciones y ciclos 13

10.7 Llamadas y retorno 13

11. Tipos de datos 13

11.1 Tipos simples 13

11.2 Tipos compuestos 13

12. Funciones 14

12.1 Escena (función con retorno) 14
12.2 Proceso (procedimiento sin retorno) 14
12.3 Llamada 14
13. Estructuras de control 15
13.1 Condicional: evaluar si / sino evaluar / sino 15
13.2 Condición múltiple: suceso 15
13.3 Ciclo controlado: rastrear 15
13.4 Ciclo condicional: seguir mientras 15
14. Operaciones 15
14.1 Aritméticas 15
14.2 Relacionales 16
14.3 Lógicas 16
15. Organización de archivos 16
15.1 Estructura de un programa .eng 16
15.2 Organización del proyecto 16
16. Importaciones 17
17. Proceso de compilación 17
18. Proceso de ejecución 18
19. Manejo de errores 18
19.1 Errores detectados 18
19.2 Formato del mensaje (ejemplo) 19
19.3 Advertencias 20
20. Innovación 21
21. Arquitectura del aplicativo 21
22. Pruebas 22
23. Contribuciones de los integrantes 27
24. Conclusiones 27
25. Referencias 27
26. Anexos 28
Anexo A. Ejemplo de código Enigma 28
Anexo B. Logotipo 29
Anexo C. Ventana principal del IDE 29
Anexo D. Casos de práctica 30
Anexo E. Retroalimentación de errores 30
Anexo F. Pruebas automatizadas 31

1. Introducción

Enigma es un lenguaje de programación de propósito educativo diseñado para entrenar y reactivar el razonamiento lógico y el pensamiento crítico. Su particularidad es que toda la sintaxis está construida sobre una narrativa de investigación detectivesca: las variables son pistas, las constantes son hechos, el programa principal es un caso y las funciones son escenas. De esta forma, programar se convierte en resolver un caso mediante deducción.

Los programas de Enigma se escriben en archivos con extensión .eng y se ejecutan desde un IDE gráfico que incluye ejecución y pruebas inmediatas. La implementación está desarrollada en Python. El elemento más innovador del lenguaje es su Analizador de Errores y Deducciones, un compilador que no solo señala que existe un error, sino que primero ofrece un indicio para que el estudiante lo encuentre y después explica en detalle qué falló en su razonamiento.

Este documento presenta el problema que motivó el proyecto, la propuesta de solución, la definición completa del lenguaje (palabras reservadas, sintaxis, semántica, tipos de datos, funciones, estructuras de control y operaciones), así como el proceso de compilación y ejecución, el manejo de errores, la arquitectura del aplicativo y las pruebas realizadas.

2. Planteamiento del problema

2.1 Problema o motivación

Con la aparición de numerosas herramientas de Inteligencia Artificial, los estudiantes han desarrollado una fuerte dependencia hacia ellas. Esto ha provocado una disminución significativa del pensamiento crítico y del razonamiento lógico en la solución de problemas complejos: en lugar de hacer el esfuerzo mental de analizar cada detalle de una situación y construir una solución propia, se delega ese trabajo a una IA.

2.2 A quiénes afecta

Afecta a estudiantes de los últimos años de colegio y a aspirantes a ingresar a la universidad.

2.3 Contexto

El problema se manifiesta en la preparación para exámenes que exigen un razonamiento lógico desarrollado, como las pruebas de aptitud académica y los exámenes de admisión. En estas pruebas no se permite el uso de calculadoras, dispositivos tecnológicos ni IA, por lo que el éxito depende totalmente de la agilidad mental y el pensamiento crítico de cada estudiante.

2.4 Soluciones existentes y sus limitaciones

Actualmente existen documentos con preguntas similares a las de los exámenes, herramientas web que recrean el entorno de la prueba y centros de preparación que brindan recursos. Sin embargo, estos métodos se basan en la repetición y la memorización. Si un estudiante realiza una práctica y falla una respuesta sin comprender por qué, seguirá practicando de forma estática: marcará respuestas sin construir una lógica previa y, por tanto, sin mejorar su razonamiento.

3. Evidencias de investigación

3.1 Análisis de soluciones existentes

Tipo de solución	Cómo funciona	Limitación identificada
Documentos de práctica	Cuadernillos con preguntas similares a las de los exámenes de admisión.	Enfoque repetitivo y memorístico; no explican el porqué de un error.
Herramientas web	Simulan el entorno del examen.	Validan la respuesta final, pero no el razonamiento seguido.
Centros de preparación	Brindan recursos y tutorías presenciales.	Dependen de horarios y costos; la retroalimentación no es inmediata.

3.2 Fuentes y evidencia

Sin duda la aparición de la inteligencia artificial fue una gran ayuda y beneficio para el desarrollo de muchos procesos, sus grandes capacidades permiten que muchas tareas se resuelvan en una cantidad de tiempo mucho menor. Al mismo tiempo estas herramientas brindan a los usuarios una mayor seguridad y una forma más ágil y rápida de resolver sus trabajos que no se lograría de otra forma (Navarro Jiménez et al., 2026).

Sin embargo, el uso constante y despreocupado de estas herramientas ha despertado diversas preocupaciones con respecto a la autonomía cognitiva y a la capacidad de razonamiento crítico de las personas. El pensamiento crítico es una de las habilidades más importantes para el aprendizaje significativo, ya que este permite analizar, cuestionar y evaluar la información de una manera más lógica y razonable. No obstante, la comodidad y la rapidez que brinda la IA puede llegar a reducir estos procesos donde las personas razonan y analizan los detalles, reemplazándolos por respuestas inmediatas que limitan la capacidad de pensar de manera autónoma, analizar críticamente la información y tomar decisiones por su propia cuenta (Navarro Jiménez et al., 2026).

Según Gerlich (2025), mediante diversos procesos realizados, lograron encontrar que los participantes más jóvenes mostraron una mayor dependencia a estas herramientas de IA en comparación con los participantes de mayor edad. Según los resultados obtenidos se logra resaltar los posibles efectos que puede tener la dependencia a estas tecnologías y la importancia de fomentar el pensamiento crítico en un mundo que está siendo impulsado por la IA.

4. Propuesta de solución

Enigma será una herramienta que, mediante una experiencia interactiva, ayude a desarrollar la lógica y el pensamiento crítico. El estudiante codificará soluciones y resolverá acertijos (tipo adivinanza o Escape Room que se resuelven programando) y casos que lo obliguen a razonar activamente. Cuando cometa un error, este será explicado para que comprenda qué falló en su lógica y pueda mejorar.

4.1 Características principales

- → IDE gráfico accesible con entorno de ejecución y pruebas inmediatas.
- → Archivos con extensión .eng.
- → Sintaxis con narrativa temática de investigación (pistas, hechos, casos, escenas).
- → Analizador de Errores y Deducciones: determina fallas lógicas en las soluciones y explica por qué el razonamiento falló.
- → Retroalimentación en dos niveles: primero un indicio para que el estudiante busque el error; después la explicación completa.

→ Desarrollado en Python.

4.2 Beneficios esperados

- ✓ Reactivación y fortalecimiento de la agilidad mental y el pensamiento crítico.
- ✓ Comprensión profunda de lo que se está realizando.
- ✓ Diagnóstico claro de las fallas de razonamiento mediante una explicación detallada de los errores.
- ✓ Aprender a programar de forma creativa usando narrativa temática.

4.3 Ejemplo de programa que puede desarrollarse

Analizador de problemas lógicos.

4.4 Relación entre el problema y el lenguaje

La siguiente tabla relaciona cada necesidad identificada en el planteamiento del problema con la característica de Enigma que la atiende y con la sección de este documento donde se describe.

Necesidad identificada	Característica de Enigma que la atiende	Dónde se describe
Dependencia de la inteligencia artificial: se delega el razonamiento y se pierde el hábito de analizar.	Retroalimentación en dos niveles: primero un indicio que orienta sin decir dónde está el error y solo después la explicación completa. El estudiante debe buscar y razonar antes de recibir la respuesta.	Secciones 19 y 20
Los exámenes de admisión exigen razonar sin calculadora ni ayuda externa.	Operaciones (conectar, descartar, intensificar, repartir) y condiciones lógicas que obligan a plantear el cálculo paso a paso; los casos de práctica entregan el reto sin la solución.	Secciones 13, 14 y 20
Las prácticas actuales se basan en repetir y memorizar, y no explican por qué falló una respuesta.	Explicación completa de cada error (tipo, archivo, línea, descripción y posible causa) y advertencias sobre código sospechoso.	Sección 19
Poca motivación ante ejercicios estáticos y repetitivos.	Sintaxis con narrativa de investigación (pistas, hechos, casos, escenas) y acertijos tipo Escape Room que se resuelven programando.	Secciones 7, 8 y 20

Necesidad identificada	Característica de Enigma que la atiende	Dónde se describe
Dificultad para pasar de la idea al código.	Guía del detective con ejemplos que se insertan en el editor y Constructor de bloques que arma instrucciones con formularios y las valida con el compilador.	Secciones 20 y 21

5. Objetivos

5.1 Objetivo general

Diseñar e implementar Enigma, un lenguaje de programación con entorno gráfico y analizador de errores explicativo, que permita a estudiantes de último año de colegio y aspirantes universitarios entrenar su razonamiento lógico y deductivo mediante la resolución de casos.

5.2 Objetivos específicos

- Definir la identidad, las palabras reservadas, la sintaxis y la semántica del lenguaje Enigma.
- Implementar las fases de análisis léxico, sintáctico y semántico en Python para archivos .eng.
- Desarrollar un IDE gráfico que permita escribir, ejecutar y probar programas de forma inmediata.
- Construir el Analizador de Errores y Deducciones que entregue primero un indicio y luego una explicación detallada del error.
- Validar el funcionamiento del lenguaje mediante casos de prueba y programas de ejemplo.

6. Alcance y limitaciones

6.1 Alcance

Lenguaje con variables, constantes, tipos simples y compuestos (pistas y tablero), condicionales, ciclos, funciones y procesos.

Importación de archivos .eng mediante la directiva #expediente.

IDE gráfico con ejecución y pruebas inmediatas.

Mensajes de error explicativos en forma de Indicio Fallido de Razonamiento.

Advertencias del compilador sobre código sospechoso que no impide ejecutar.

Público objetivo: estudiantes de último año de colegio y aspirantes a la universidad.

6.2 Limitaciones

Enigma es un lenguaje educativo; no está pensado para desarrollo de software de producción.

Las variables son de tipado estático y los tipos no pueden mezclarse dentro de una lista o un tablero.

Las cadenas (expresion) no permiten modificar un solo carácter; se debe reemplazar el texto completo.

Los valores decimales no pueden usarse como contador en ciclos rastrear.

Todo archivo .eng, incluso uno que solo se importa, debe contener exactamente un bloque caso.

No sustituye el estudio de las pruebas de admisión; es un complemento para desarrollar razonamiento.

7. Descripción del lenguaje

7.1 Identidad

Aspecto	Definición
Nombre	Enigma
Propósito	Entrenar y reactivar la capacidad de razonamiento lógico y deductivo para exámenes mediante la resolución de problemas.
Usuarios	Estudiantes de último año de colegio y aspirantes a las universidades.
Extensión de archivos	.eng
Logotipo	Un dado (cubo con símbolos ?, ! y Σ), mostrado en la portada.
Filosofía de diseño	“Todo dato es una pista. Toda operación es un paso más. Todo programa es un caso que se resuelve con lógica, razonar es construir.”
Paradigma	Imperativo y procedural, con tipado estático.
Implementación	Python, con IDE gráfico.

7.2 Ejemplo de código

El siguiente programa, que corresponde al proyecto de ejemplo ejemplos/investigacion_caso del repositorio, importa la biblioteca analisis.eng, declara una constante, define un proceso que consulta a esa biblioteca si el sospechoso es mayor de edad y, en el caso principal, recorre tres rondas de interrogatorio:

```
#expediente <analisis.eng>

hecho MAX_INTENTOS = entero(valor: 3)!

proceso VerificarSospechoso(nombre: expresion, edad: entero) inicio
    evaluar si (resolver EsMayorDeEdad(edad)) inicio
        revelar('El sospechoso es mayor de edad: ', nombre)!
    fin!
    sino inicio
        revelar('El sospechoso es menor de edad: ', nombre)!
    fin!
fin!

caso InvestigacionCaso abierto
    pista sospechoso1 = expresion(valor: 'Carlos')!
```

```
    pista edadSospechoso = entero(valor: 25)!
    pista contador = entero(valor: 1)!
    seguir mientras (contador <= MAX_INTENTOS) inicio
        revelar('Ronda de interrogatorio numero: ', contador)!
        contador = conectar(contador, 1)!
    fin!
    resolver VerificarSospechoso(sospechoso1, edadSospechoso)!
caso InvestigacionCaso cerrado!
```

8. Palabras reservadas

Las palabras reservadas no pueden usarse como nombres de variables, constantes, escenas o procesos. Se agrupan por categoría:

Categoría	Palabra	Propósito
Declaración	pista	Declarar una variable.
	hecho	Declarar una constante.
Programa	caso	Programa principal (equivalente a main).
	abierto / cerrado	Apertura y cierre del bloque caso.
Entrada/Salida	revelar	Mostrar en pantalla.
	consultar	Leer datos del usuario.
Condicionales	evaluar si	Condicional de deducción (if).
	sino evaluar	Alternativa deductiva (else if).
	sino	Alternativa deductiva (else).
	suceso	Condición múltiple (switch).
Ciclos	rastrear … desde … hasta	Ciclo for, repetición controlada.
	seguir mientras	Ciclo while.
Bloques	inicio / fin!	Delimitan el inicio y el fin de un bloque.
Funciones	escena	Definir una función que devuelve un valor.
	proceso	Definir un procedimiento que no devuelve valor.

Categoría	Palabra	Propósito
	resolver	Llamar a una escena o proceso.
	concluir	Retornar un valor (return).
Operaciones	conectar	Suma.
	descartar	Resta.
	intensificar	Multiplicación.
	repartir	División.
Tipos	entero, decimal, expresion, declaracion	Tipos simples.
	pistas, tablero	Tipos compuestos (arreglo y matriz).
Valores	culpable / NoCulpable	Booleano verdadero / falso.
	sin_pista	Valor y tipo nulo.
Lógicos	y, o, no	AND, OR y NOT.
Importación	#expediente	Importar funciones de otro archivo.

9. Sintaxis

Tabla completa de elementos sintácticos del lenguaje, con su forma de uso y un ejemplo.

Palabra	Propósito	Forma de uso	Ejemplo
pista	Declarar variable	pista <nombre> = <tipo>(valor: x)!	pista sospechoso1 = entero(valor: 15)!
hecho	Declarar constante	hecho <nombre> = <tipo>(valor: x)!	hecho sospechoso2 = entero(valor: 10)!
!	Terminador de instrucción	<instrucción>!	pista s1 = entero(valor: 15)!
caso	Programa principal	caso Nombre abierto … … caso Nombre cerrado!	caso AreaCirculo abierto … … caso AreaCirculo cerrado!

Palabra	Propósito	Forma de uso	Ejemplo
revelar	Salida en pantalla	revelar('texto', variable)!	revelar('El sospechoso es: ', sospechoso1)!
consultar	Lectura de datos	consultar(<variable>)!	consultar(valorPremisa)!
evaluar si	Condicional (if)	evaluar si (<cond>) inicio … fin!	evaluar si (pista1 > pista2) inicio … fin!
sino evaluar	Else if	sino evaluar (<cond>) inicio … fin!	sino evaluar (pista1 < pista2) inicio … fin!
sino	Else	sino inicio … fin!	sino inicio … fin!
suceso	Condición múltiple	suceso (variable) inicio 1 (cond): … fin!	suceso (edad) inicio 1 (edad < 18): revelar('Menor')! … fin!
rastrear	Ciclo for	rastrear <var> desde X hasta Y inicio … fin!	rastrear pasos desde 1 hasta 10 inicio … fin!
seguir mientras	Ciclo while	seguir mientras (<cond>) inicio … fin!	seguir mientras (pista1 > pista2) inicio … fin!
escena	Función con retorno	escena Nombre(args)::tipo inicio … fin!	escena BuscarEnLista(nombre: expresion)::expresion inicio … fin!
proceso	Procedimiento	proceso Nombre(args) inicio … fin!	proceso ModificarLista(lista: pistas) inicio … fin!
resolver	Llamar función	resolver Nombre(argumentos)!	resolver BuscarEnLista(nombre)!
concluir	Retorno (return)	concluir <expresión>!	concluir culpable!
conectar	Suma	conectar(p1, p2, …)	conectar(pista1, pista2, pista3)
descartar	Resta	descartar(p1, p2)	descartar(pista1, pista2)
intensificar	Multiplicación	intensificar(p1, p2)	intensificar(pista1, pista2)
repartir	División	repartir(p1, p2)	repartir(pista1, pista2)
pistas	Arreglo	pista var = pistas tipo [A, B, C]	pista archivos = pistas entero [2, 4, 6]

Palabra	Propósito	Forma de uso	Ejemplo
tablero	Matriz	pista var = tablero tipo [filas][columnas]	pista Celda = tablero entero [4][4]
o / y / no	Operadores lógicos	(cond) o (cond) · (cond) y (cond) · no (cond)	(pista1 == arma) o (pista1 == libro)
== <> > < >= <=	Comparación	variable <op> valor	edad >= 20
{ }	Comentario	{texto del comentario}	{esto es un comentario}
#expediente	Importar	#expediente <archivo.eng>	#expediente <analisis.eng>

10. Semántica

10.1 Delimitador de instrucción

El signo ! sustituye al punto y coma de otros lenguajes. Se usa exclusivamente al final de las sentencias ejecutables (declaración de pista y hecho, consultar, revelar, asignaciones, resolver y concluir) y al finalizar bloques, rutinas y el programa principal (fin!, caso < Nombre > cerrado!). No llevan ! los encabezados o inicios de bloque (inicio, abierto) ni las directivas de importación.

10.2 Inicio del programa y bloques

La ejecución comienza buscando obligatoriamente el bloque caso. Todo archivo ejecutable debe contener exactamente uno. Los bloques de control, ciclos y funciones se delimitan con inicio y fin!. La indentación no es obligatoria para el compilador, pero el IDE la recomienda para mantener la legibilidad narrativa.

10.3 Variables y constantes

Las variables son de tipado estático. Se declaran con pista, un identificador y un tipo inicial con valor explícito o indeterminado. Para reasignar basta invocar el nombre y asignar una expresión compatible. Las constantes se declaran con hecho; intentar modificarlas genera un error de compilación.

10.4 Alcance (scope)

Global: las pistas declaradas fuera de escenas, procesos o casos son accesibles en todo el archivo.

Local: las variables o parámetros declarados dentro de una escena, proceso, evaluar si, rastrear o seguir mientras solo existen entre inicio y fin de ese bloque.

10.5 Evaluación de expresiones

Las operaciones aritméticas (conectar, descartar, intensificar, repartir) se evalúan respetando el orden de los argumentos. Las operaciones relacionales y lógicas (y, o, no) evalúan las condiciones de izquierda a derecha.

10.6 Condiciones y ciclos

Las estructuras condicionales y de iteración evalúan una condición lógica que determina si el bloque entre inicio y fin se ejecuta o se omite. El ciclo rastrear usa un iterador finito que se incrementa de uno en uno desde el valor inicial hasta el límite final.

10.7 Llamadas y retorno

Se invocan con resolver seguido del nombre y los argumentos.

Una escena debe retornar obligatoriamente una expresión compatible con su tipo mediante concluir; el compilador comprueba que toda escena tenga al menos un concluir.

Un proceso ejecuta sus instrucciones y termina en fin!; no permite concluir.

11. Tipos de datos

11.1 Tipos simples

Tipo	Declaración	Uso	Operaciones	Restricciones
entero	pista1 = entero(valor: 15)	Números enteros (positivos o negativos) para conteos.	Aritméticas y relacionales.	No acepta decimales.
decimal	pista2 = decimal(valor: 15.5)	Cantidades continuas o medidas con precisión decimal.	Aritméticas y relacionales.	Ocupa más memoria que entero; no puede ser contador de rastrear.
expresion	pista3 = expresion(valor: 'Estaba en el baño.')	Cadenas de texto entre comillas simples.	revelar, consultar, comparación (==, <>).	No se puede modificar un solo carácter; se reemplaza el texto completo.
declaracion	pista4 = declaracion(valor: culpable)	Estados de verdad lógica.	Lógicas (y, o, no) e igualdad/diferencia.	Solo culpable o NoCulpable.
sin_pista	pista5 = entero(valor: sin_pista)	Ausencia explícita de valor o premisa no resuelta.	Comparación == / <> sin_pista.	No se opera aritméticamente ni se evalúa en condiciones hasta reasignarlo.

11.2 Tipos compuestos

Tipo	Declaración	Uso	Operaciones	Restricciones

Tipo	Declaración	Uso	Operaciones	Restricciones
pistas (lista / arreglo)	pista listaEnteros = pistas entero [2, 4, 6]	Colección secuencial ordenada de elementos del mismo tipo simple.	Paso como parámetro, lectura de elementos y modificación dentro de procesos.	No se pueden mezclar tipos.
tablero (matriz)	pista Celda = tablero entero [4][4]	Datos en filas y columnas (cuadrícula, mapa, tabla).	Acceso por índices, recorrido con rastrear anidados, modificación de celdas.	Tamaños obligatorios al crearlo; no se mezclan tipos.

12. Funciones

Enigma distingue dos tipos de subrutinas, según devuelvan o no un valor:

12.1 Escena (función con retorno)

Se define con escena Nombre(argumentos)::tipo inicio … fin!. Debe terminar devolviendo un valor compatible con el tipo declarado mediante concluir.

```
escena SumarPistas(a: entero, b: entero)::entero inicio
    concluir conectar(a, b)!
fin!
```

12.2 Proceso (procedimiento sin retorno)

Se define con proceso Nombre(argumentos) inicio … fin!. Ejecuta sus instrucciones y finaliza en fin!; no admite concluir. Puede recibir listas y modificarlas.

12.3 Llamada

Ambas se invocan con resolver Nombre(argumentos)!. Los parámetros y variables declarados dentro de la subrutina son locales.

Una escena puede usarse como valor: pista r = resolver SumarPistas(2, 3)! guarda en r el resultado de la escena. Un proceso, en cambio, no devuelve ningún valor y por eso no puede usarse de esa manera: el compilador lo señala como error semántico.

13. Estructuras de control

13.1 Condicional: evaluar si / sino evaluar / sino

```
evaluar si (pista1 > pista2) inicio
   revelar('pista1 pesa más')!
fin!
sino evaluar (pista1 < pista2) inicio
   revelar('pista2 pesa más')!
fin!
sino inicio
   revelar('Las pistas son iguales')!
fin!
```

13.2 Condición múltiple: suceso

```
suceso (edad) inicio
   1 (edad < 18): revelar('El sospechoso es menor')!
   2 (edad > 18): revelar('El sospechoso es mayor')!
   3 (edad == 18): revelar('El sospechoso tiene 18')!
fin!
```

13.3 Ciclo controlado: rastrear

```
rastrear pasos desde 1 hasta 10 inicio
   revelar('Paso número: ', pasos)!
fin!
```

13.4 Ciclo condicional: seguir mientras

```
seguir mientras (contador <= 3) inicio
   contador = conectar(contador, 1)!
fin!
```

14. Operaciones

14.1 Aritméticas

Operación	Símbolo equivalente	Ejemplo
conectar	+	conectar(pista1, pista2, pista3)
descartar	-	descartar(pista1, pista2)
intensificar	*	intensificar(pista1, pista2)
repartir	/	repartir(pista1, pista2)

14.2 Relacionales

Operador	Significado	Ejemplo
==	Igual	edad == 20
<>	Diferente	edad <> 20
>	Mayor	edad > 20
<	Menor	edad < 20
>=	Mayor o igual	edad >= 20
<=	Menor o igual	edad <= 20

14.3 Lógicas

Operador	Significado	Ejemplo
y	AND	(pista1 == arma) y (pista1 == libro)
o	OR	(pista1 == arma) o (pista1 == libro)
no	NOT (negación)	no (pista1 == arma)

15. Organización de archivos

15.1 Estructura de un programa .eng

Un archivo Enigma se organiza, de arriba hacia abajo, en el siguiente orden:

Importaciones (#expediente) al inicio del archivo.

Constantes y variables globales (hecho, pista).

Definiciones de escenas y procesos.

El bloque principal caso … abierto / cerrado! (exactamente uno por archivo ejecutable).

15.2 Organización del proyecto

La organización real de los archivos de Enigma separa el núcleo del lenguaje, la interfaz gráfica, los ejemplos y las pruebas:

```
enigma/
├── main.py
├── core/
│   ├── tokens.py
│   ├── lexer.py
```

```
│   ├── ast_nodes.py
│   ├── parser.py
│   ├── parser_base.py
│   ├── parser_sentencias.py
│   ├── parser_control.py
│   ├── parser_expresiones.py
│   ├── parser_funciones.py
│   ├── module_resolver.py
│   ├── symbols.py
│   ├── semantic.py
│   ├── advertencias.py
│   ├── interpreter.py
│   └── errors.py
├── gui/
│   ├── main_window.py
│   ├── editor_widget.py
│   ├── file_tree.py
│   ├── output_panel.py
│   ├── panel_derecho.py
│   ├── guia_rapida.py
│   ├── guia_detective.py
│   ├── guia_contenido.py
│   ├── constructor_bloques.py
│   ├── constructor_codigo.py
│   └── indicios.py
├── ejemplos/
├── ejemplo_gui/
├── herramientas/
└── tests/
    └── programas_prueba/
```

Esta separación permite mantener independiente la lógica del lenguaje de su presentación. Los archivos del núcleo pueden probarse sin abrir Tkinter, mientras que la carpeta gui administra proyectos, editor, árbol de archivos, salida, guía y constructor visual de código.

16. Importaciones

La directiva #expediente < archivo.eng > importa módulos o archivos externos completos. Se declara al inicio del programa, fuera de la estructura caso. Antes de compilar o ejecutar el código principal, el sistema carga e integra todas las definiciones externas en el espacio de trabajo global. La directiva no lleva el terminador !.

```
#expediente <analisis.eng>
```

17. Proceso de compilación

Al compilar un archivo .eng, el sistema recorre las siguientes fases:

Análisis léxico: el texto se divide en tokens (palabras reservadas, identificadores, literales, operadores, comentarios).

Análisis sintáctico: se verifica la estructura (bloques inicio/fin!, terminador !, un único caso).

Resolución de importaciones: se leen las directivas #expediente, se integran los archivos referenciados y se detectan archivos inexistentes, funciones repetidas e importaciones circulares.

Análisis semántico: se validan tipos, alcance, constantes no reasignadas, valores sin_pista, llamadas a funciones y que toda escena tenga su concluir.

Advertencias: sobre el árbol ya analizado se buscan situaciones sospechosas que no impiden ejecutar (pista declarada y no usada, parámetro sin usar, código después de concluir). Se muestran junto con el resultado, incluso cuando hay un error (ver sección 19.3).

Analizador de Errores y Deducciones: si hay fallas, se genera un Indicio Fallido de Razonamiento con línea y explicación (ver sección Manejo de errores).

Enigma utiliza un intérprete de árbol escrito directamente en Python. No genera código Python ni código de máquina. El proceso consta de: lexer, parser, resolución de importaciones, análisis semántico e interpretación del AST.

El proyecto no utiliza PLY, Lark ni ANTLR. Los análisis léxico y sintáctico están implementados manualmente en los módulos correspondientes de core. Esta decisión facilita relacionar cada fase con los conceptos estudiados en el curso y controlar los mensajes educativos de los errores.

18. Proceso de ejecución

Una vez compilado sin errores, la ejecución comienza en el bloque caso. Las instrucciones se ejecutan de forma secuencial; revelar muestra resultados en la consola del IDE y consultar solicita datos al usuario. Las llamadas con resolver transfieren el control a la escena o proceso correspondiente y regresan al terminar. El IDE permite ejecutar y probar el programa de forma inmediata. Durante la ejecución se detectan los errores que dependen de los valores reales, como dividir entre cero, acceder fuera de los límites de una lista o un tablero, o escribir un dato que no corresponde al tipo esperado en consultar.

Desde el IDE, el usuario puede compilar con el botón Compilar o ejecutar con Ejecutar caso, disponible en la barra del editor. También están definidos los atajos Ctrl+Enter para ejecutar y compilar, y Ctrl+O para abrir una carpeta de proyecto.

Al ejecutar, el intérprete inicia en el bloque caso... abierto y muestra los resultados en el panel inferior de salida. La entrada solicitada mediante consultar se introduce en el terminal integrado. Los errores de compilación se muestran con el formato de Indicio Fallido de Razonamiento, mientras que la salida correcta permanece visible y separada de los mensajes de error.

19. Manejo de errores

Enigma no genera excepciones genéricas. Cuando detecta una inconsistencia produce un Indicio Fallido de Razonamiento que indica la línea y la falla lógica cometida. El aprendizaje se guía en dos niveles: primero un indicio para que el estudiante localice el error por sí mismo, y luego la explicación completa.

19.1 Errores detectados

Situación	Ejemplo
Operar matemáticamente un valor sin_pista	conectar(pista5, 2) con pista5 = sin_pista

Situación	Ejemplo
Incompatibilidad de tipos (entero vs expresion)	Asignar ‘hola’ a una pista entero
Rebasar las dimensiones de un tablero (se detecta al ejecutar)	Celda[5][5] en un tablero entero[4][4]
Falta el terminador ! en una sentencia	revelar('hola') (sin !)
Falta el cierre de un bloque	Un inicio sin su fin!
Reasignar una constante (hecho)	MAX_INTENTOS = entero(valor: 5)!
Escena sin concluir	escena Vacia()::entero inicio … fin! sin ningún concluir
Palabra reservada usada como nombre	pista y = entero(valor: 1)! (y es el operador lógico)

19.2 Formato del mensaje (ejemplo)

El mensaje se muestra en el panel de salida en dos pasos. La primera vez que aparece el error, el IDE entrega solo un indicio que orienta sin decir dónde está el problema (Figura 19.1):

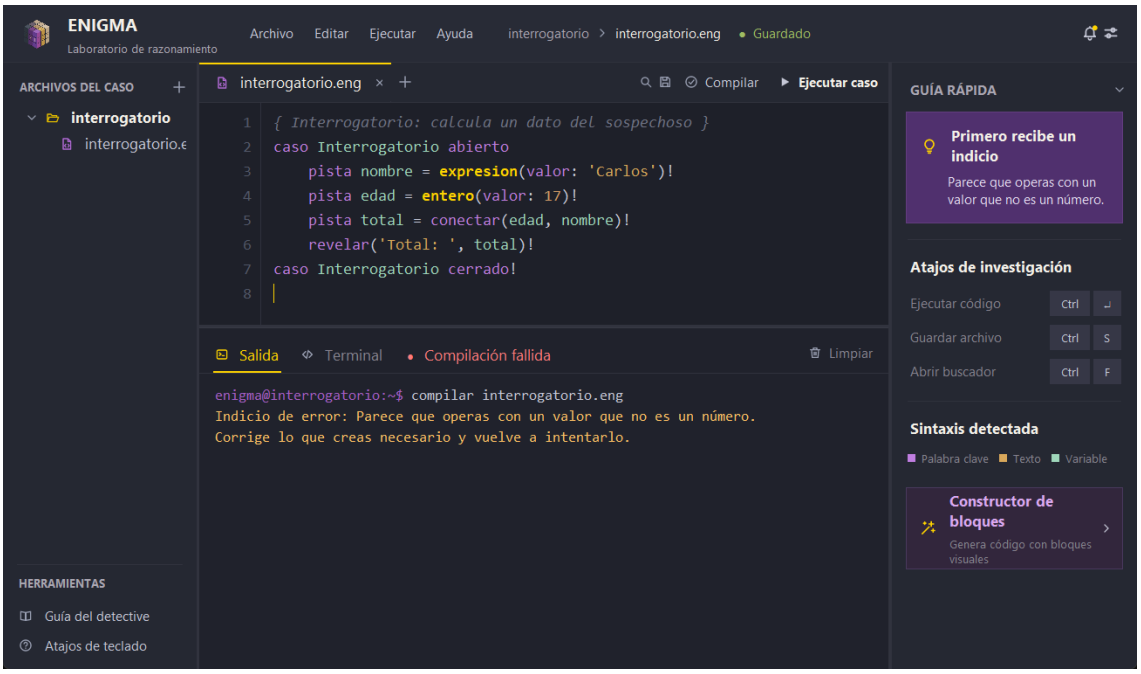


Figura 19.1. Primer paso: el IDE muestra solo un indicio del error.

Si se vuelve a compilar sin corregir el error, aparece la explicación completa, con el tipo de error, el archivo, la línea, la descripción y la posible causa (Figura 19.2):

Error de semántica
Archivo: interrogatorio.eng
Línea: 5
Descripción: No se puede aplicar 'conectar' a un valor de tipo 'expresion': solo 'entero' y 'decimal' participan en operaciones aritméticas.
Posible causa: Se usó un valor no numérico en una operación.

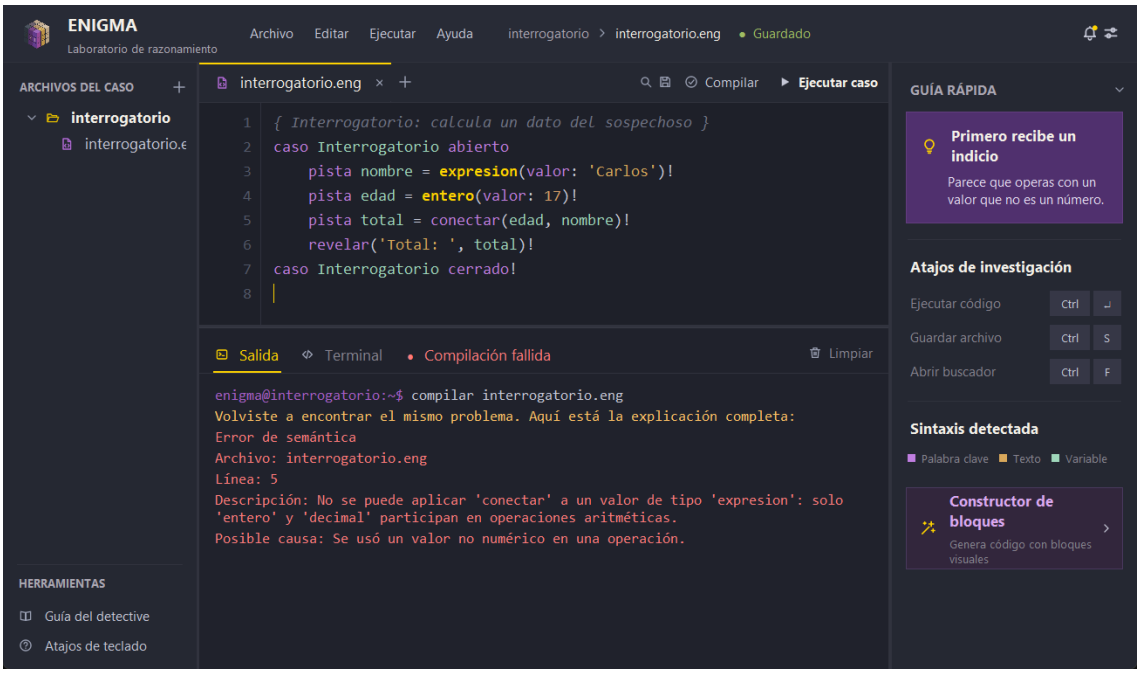


Figura 19.2. Segundo paso: la explicación completa del mismo error.

19.3 Advertencias

Además de los errores, el compilador emite advertencias: avisos sobre código válido pero sospechoso, que no impiden compilar ni ejecutar. Se calculan sobre el árbol sintáctico ya analizado y se muestran en el panel de salida en color azul, para no confundirlas con los errores.

Advertencia	Cuándo se avisa	Ejemplo
Pista sin usar	Una pista se declara y nunca se usa.	pista olvidada = entero(valor: 2)!
Parámetro sin usar	Una escena o un proceso no usa uno de sus parámetros.	escena Primero(uno: entero, dos: entero) que solo usa uno
Código inalcanzable	Hay instrucciones después de un concluir en el mismo bloque.	concluir uno! seguido de revelar('…')!

Siguen la misma idea de dos pasos que los errores: la primera vez que se compila se muestra un resumen por tipo, por ejemplo «2 parámetros que nunca se usan», sin decir dónde; si se vuelve a compilar sin cambiarlas, se muestra el detalle con archivo, línea, descripción y posible causa. También se muestran cuando hay un error de compilación, para que el estudiante vea todo lo que debe revisar.

20. Innovación

La innovación de Enigma es el Analizador de Errores y Deducciones. A diferencia de las herramientas de práctica tradicionales, que solo indican si la respuesta fue correcta o incorrecta, este compilador determina las fallas lógicas en la resolución de los ejercicios y explica en detalle por qué el razonamiento falló.

Retroalimentación gradual: antes de mostrar el error completo se entrega un indicio para que el estudiante lo busque.

Enfoque en el razonamiento: el objetivo es que el estudiante construya lógica, no que memorice respuestas.

Narrativa temática: la sintaxis detectivesca convierte la programación en la resolución de un caso, con acertijos tipo adivinanza o Escape Room.

Diseño sin ayudas externas: promueve el esfuerzo mental que exigen los exámenes de admisión.

Advertencias en dos pasos: la misma idea de indicio primero y detalle después se aplica a lo sospechoso que no es un error (ver sección 19.3).

Además de los acertijos completos, la Guía del detective incluye una sección de Casos de práctica. Estos retos presentan una situación narrativa, el objetivo esperado y un esqueleto de código editable, pero no incluyen la solución completa. Actualmente incluyen casos de dificultad creciente:

- Abrir una caja fuerte.
- Encontrar al culpable mediante pistas y deducción.
- Resolver la suma del cofre.
- Abrir el tablero sellado, combinando estructuras iterativas y matrices.

Al insertar un caso en el editor, el usuario puede elegir el nombre de la carpeta principal y su ubicación. El IDE permite mantener varias carpetas principales abiertas, administrar archivos y subcarpetas, renombrar o eliminar elementos, guardar y utilizar Guardar como….

El Constructor de bloques permite armar instrucciones con formularios y validarlas con el compilador antes de insertarlas. Escribe el bloque en el archivo abierto o, si no hay ninguno, crea un proyecto y un archivo nuevos, igual que los ejemplos de la Guía del detective.

21. Arquitectura del aplicativo

El aplicativo está desarrollado en Python y se compone de un IDE gráfico y de un núcleo de compilación/ejecución. Se propone la siguiente organización lógica de módulos:

Módulo	Responsabilidad
Ventana principal	Coordina el menú, el editor, el árbol de archivos y los paneles laterales.
Árbol de archivos	Administra proyectos múltiples, carpetas, archivos, renombrado, eliminación y expansión de directorios.

Módulo	Responsabilidad
Editor de código	Mantiene pestañas, rutas, cambios sin guardar, guardado, Guardar como y resaltado de sintaxis.
Panel de salida	Muestra compilación, ejecución, terminal de entrada y errores.
Guía del detective	Presenta la documentación del lenguaje, ejemplos, acertijos y casos de práctica.
Constructor de bloques	Permite formar instrucciones mediante controles visuales.
Analizador léxico (lexer.py)	Convierte el texto fuente en tokens.
Analizador sintáctico (parser*.py)	Construye el árbol sintáctico y valida la estructura.
Analizador semántico (semantic.py)	Valida tipos, alcance, constantes, llamadas a funciones y retornos.
Advertencias (advertencias.py)	Detecta código sospechoso que no impide ejecutar: pistas y parámetros sin usar, código después de concluir.
Analizador de Errores y Deducciones (gui/indicios.py, errores del núcleo)	Traduce cada falla a un Indicio Fallido de Razonamiento con indicio y explicación.
Ejecutor (interpreter.py)	Ejecuta el programa a partir del bloque caso.
Gestor de importaciones (module_resolver.py)	Resuelve las directivas #expediente.

Flujo general: Código .eng → Léxico → Sintáctico → Importaciones → Semántico → (errores → Analizador de Deducciones) / (sin errores → Ejecutor) → Consola del IDE. Las advertencias se calculan sobre el árbol ya analizado y se muestran junto con el resultado de la compilación, incluso cuando hay un error.

La implementación mantiene separadas las responsabilidades de los módulos. Por ejemplo, main.py inicia VentanaPrincipal; esta clase integra ArbolArchivos, EditorCodigo, PanelSalida y PanelDerecho. El editor entrega el contenido al núcleo, que realiza todas las fases de procesamiento antes de ejecutar el AST.

22. Pruebas

Las pruebas automatizadas se implementaron con pytest y cubren el lexer, el parser, el análisis semántico, el intérprete, las importaciones, los casos narrativos y las funciones principales del IDE. La ejecución completa del proyecto aprobó 378 pruebas.

El enunciado exige documentar veinte programas de prueba: cinco válidos, cinco con errores sintácticos, cinco con errores semánticos, tres de importaciones, una de importación circular y una prueba completa relacionada con el problema. Están en la carpeta tests/programas_prueba/ y se ejecutan con pytest. El identificador indica el tipo de prueba (V = válido, S = sintáctico, M = semántico, I = importación, C = importación circular, P = prueba completa):

#	Objetivo	Entrada	Resultado esperado	Resultado obtenido	Estado
V1	Declarar variables y constantes, usar las cuatro operaciones aritméticas y leer un dato con consultar.	validos/V1_aritmetica.eng (se escribe: 4)	Compila y se ejecuta. Imprime: Suma: 16 / Resta: 8 / Producto: 48 / Division: 3.0 / Detective: Holmes	Compila y se ejecuta. Imprime: Suma: 16 / Resta: 8 / Producto: 48 / Division: 3.0 / Detective: Holmes	Aprobado
V2	Ejecutar un condicional simple, uno con alternativas (sino evaluar, sino) y uno múltiple (suceso).	validos/V2_condicionales.eng	Compila y se ejecuta. Imprime: Puede entrar / Justo 18 / Opcion 3	Compila y se ejecuta. Imprime: Puede entrar / Justo 18 / Opcion 3	Aprobado
V3	Ejecutar un ciclo rastrear (for) y un ciclo seguir mientras (while).	validos/V3_ciclos.eng	Compila y se ejecuta. Imprime: Suma de 1 a 5: 15 / Cuenta regresiva 3 / Cuenta regresiva 2 / Cuenta regresiva 1	Compila y se ejecuta. Imprime: Suma de 1 a 5: 15 / Cuenta regresiva 3 / Cuenta regresiva 2 / Cuenta regresiva 1	Aprobado
V4	Definir y llamar una escena (con retorno) y un proceso (sin retorno) con parámetros.	validos/V4_funciones.eng	Compila y se ejecuta. Imprime: Cuadrado de 7: 49 / Detective Watson listo	Compila y se ejecuta. Imprime: Cuadrado de 7: 49 / Detective Watson listo	Aprobado
V5	Usar una lista, un tablero, los operadores lógicos y el valor nulo sin_pista.	validos/V5_compuestos_y_logica.eng	Compila y se ejecuta. Imprime: Lista: 2 40 6 / Tablero: 7 9 / b es falso / Al menos uno es verdadero / No son ambos verdaderos / Aun sin pista	Compila y se ejecuta. Imprime: Lista: 2 40 6 / Tablero: 7 9 / b es falso / Al menos uno es verdadero / No son ambos verdaderos / Aun sin pista	Aprobado

#	Objetivo	Entrada	Resultado esperado	Resultado obtenido	Estado
S1	Detectar una sentencia sin el terminador ! y señalar la línea donde falta.	sintacticos/S1_falta_terminador.eng	Error sintáctico en S1_falta_terminador.eng, línea 3	Error sintáctico en S1_falta_terminador.eng, línea 3: Se esperaba SELLO pero se encontró CASO ('caso').	Aprobado
S2	Detectar un bloque inicio al que le falta su fin!.	sintacticos/S2_bloque_sin_fin.eng	Error sintáctico en S2_bloque_sin_fin.eng, línea 6	Error sintáctico en S2_bloque_sin_fin.eng, línea 6: No se esperaba el token CASO ('caso') al inicio de una sentencia.	Aprobado
S3	Detectar un paréntesis que no se cierra.	sintacticos/S3_parentesis_sin_cerrar.eng	Error sintáctico en S3_parentesis_sin_cerrar.eng, línea 3	Error sintáctico en S3_parentesis_sin_cerrar.eng, línea 3: Se esperaba PARENTESIS_DER pero se encontró SELLO ('!').	Aprobado
S4	Detectar un condicional mal escrito (evaluar sin la palabra si).	sintacticos/S4_evaluar_sin_si.eng	Error sintáctico en S4_evaluar_sin_si.eng, línea 4	Error sintáctico en S4_evaluar_sin_si.eng, línea 4: Se esperaba SI pero se encontró PARENTESIS_IZQ ('(').	Aprobado
S5	Detectar un caso principal que se abre y nunca se cierra.	sintacticos/S5_caso_sin_cierre.eng	Error sintáctico en S5_caso_sin_cierre.eng, línea 3	Error sintáctico en S5_caso_sin_cierre.eng, línea 3: Se esperaba CASO pero se encontró FIN_DE_ARCHIVO ('').	Aprobado
M1	Detectar el uso de una variable que no fue declarada.	semanticos/M1_variable_no_declarada.eng	Error semántico en M1_variable_no_declarada.eng, línea 3	Error semántico en M1_variable_no_declarada.eng, línea 3: No se encontró ninguna variable llamada 'sospechoso' en este alcance.	Aprobado

#	Objetivo	Entrada	Resultado esperado	Resultado obtenido	Estado
M2	Detectar la reasignación de una constante declarada con hecho.	semanticos/M2_reasignar_hecho.eng	Error semántico en M2_reasignar_hecho.eng, línea 4	Error semántico en M2_reasignar_hecho.eng, línea 4: 'maximo' fue declarada con 'hecho' en la línea 3.	Aprobado
M3	Detectar la asignación de un texto a una pista de tipo entero.	semanticos/M3_tipos_incompatibles.eng	Error semántico en M3_tipos_incompatibles.eng, línea 4	Error semántico en M3_tipos_incompatibles.eng, línea 4: 'edad' es de tipo 'entero' pero se le asignó 'expresion'.	Aprobado
M4	Detectar una pista declarada dos veces en el mismo alcance.	semanticos/M4_declaracion_duplicada.eng	Error semántico en M4_declaracion_duplicada.eng, línea 4	Error semántico en M4_declaracion_duplicada.eng, línea 4: 'huella' ya fue declarada en la línea 3 dentro del ámbito 'A'.	Aprobado
M5	Detectar una operación aritmética sobre un valor sin_pista.	semanticos/M5_operar_sin_pista.eng	Error semántico en M5_operar_sin_pista.eng, línea 4	Error semántico en M5_operar_sin_pista.eng, línea 4: No se puede aplicar 'conectar' a un valor de tipo 'sin_pista': solo 'entero' y 'decimal' participan en operaciones aritméticas.	Aprobado
I1	Detectar la importación de un archivo que no existe.	importacion/I1_archivo_inexistente/principal.eng	Error de importación en principal.eng, línea 2	Error de importación en principal.eng, línea 2: El archivo 'herramientas.eng' no existe en 'tests/programas_prueba/importacion\I1_archivo_inexistente'.	Aprobado
I2	Detectar una llamada a una función importada con la cantidad de argumentos incorrecta.	importacion/I2_argumentos_incorrectos/principal.eng	Error semántico en principal.eng, línea 5	Error semántico en principal.eng, línea 5: 'Sumar' espera 2 argumento(s) pero se enviaron 3.	Aprobado

#	Objetivo	Entrada	Resultado esperado	Resultado obtenido	Estado
I3	Detectar un error dentro de un archivo importado e indicar el archivo donde ocurre.	importacion/I3_error_en_importado/principal.eng	Error semántico en calculos.eng, línea 2	Error semántico en calculos.eng, línea 2: No se encontró ninguna variable llamada 'factor' en este alcance.	Aprobado
C1	Detectar una importación circular (a.eng importa b.eng y b.eng importa a.eng).	importacion/C1_circular/principal.eng	Error de importación en b.eng, línea 1	Error de importación en b.eng, línea 1: La cadena de importaciones principal.eng -> a.eng -> b.eng -> a.eng vuelve a un archivo ya en proceso.	Aprobado
P1	Resolver un caso de deducción con varios archivos: importación, tablero, ciclo, funciones, lógica, valor nulo y lectura de la respuesta del estudiante.	completa/P1_caso_museo/principal.eng (se escribe: 1)	Compila y se ejecuta. Imprime: Una obra desaparecio del museo. Hay tres sospechosos. / Sospechoso 0 : descartado / Sospechoso 1 : culpable / Sospechoso 2 : descartado / Culpables encontrados: 1 / Quien crees que es el culpable? / Correcto: has resuelto el caso.	Compila y se ejecuta. Imprime: Una obra desaparecio del museo. Hay tres sospechosos. / Sospechoso 0 : descartado / Sospechoso 1 : culpable / Sospechoso 2 : descartado / Culpables encontrados: 1 / Quien crees que es el culpable? / Correcto: has resuelto el caso.	Aprobado

El resultado esperado se dedujo de las reglas del lenguaje antes de ejecutar cada programa. La tabla se genera ejecutando los veinte programas y comparando lo obtenido con lo esperado (python herramientas/informe_pruebas.py).

Las pruebas adicionales verifican cuatro casos de práctica, dos acertijos narrativos completos, la creación de proyectos, la ubicación predeterminada, los archivos de la guía y la actualización de rutas en el editor.

Comandos utilizados:

```
python -m pytest -q
python herramientas/informe_pruebas.py
```

23. Contribuciones de los integrantes

Integrante	Contribución
Alisson Cordero Fonseca	Diseño funcional y gráfica del proyecto, pruebas de interacción, integración educativa, gestión de proyectos en el IDE y documentación.
Melany Céspedes Sánchez	Diseño del lenguaje, núcleo de compilación e interpretación, integración del IDE, casos narrativos y documentación técnica.

24. Conclusiones

Enigma propone una alternativa distinta a los métodos de preparación basados en la repetición, al centrar el aprendizaje en el razonamiento y en la comprensión del error.

La narrativa detectivesca (pistas, hechos, casos, escenas) hace que la sintaxis sea memorable y motive al estudiante a deducir en lugar de memorizar.

El Analizador de Errores y Deducciones, con su retroalimentación en dos niveles, es el principal aporte del proyecto.

El diseño aplica conceptos del curso de Paradigmas de Programación: tipado estático, alcance, subrutinas, estructuras de control y análisis léxico, sintáctico y semántico.

Se logró desarrollar un lenguaje de programación completo y un IDE funcional que integra un lexer, parser, análisis semántico e intérprete. Asimismo, incorporó acertijos narrativos y casos de práctica que permiten que el estudiante escriba y pruebe sus propias soluciones.

Durante el desarrollo surgieron dificultades relacionadas con la sincronización entre el editor, el árbol de archivos, las rutas abiertas y la ejecución de archivos dentro de proyectos. Estas situaciones exigieron fortalecer la organización del código, separar mejor las responsabilidades y verificar cada cambio mediante pruebas automatizadas.

Como aprendizaje principal, el proyecto permitió aplicar conceptos de paradigmas y compiladores en una solución integral, y comprender que el diseño de un lenguaje incluye tanto su sintaxis y semántica como la experiencia de aprendizaje de sus usuarios. Como trabajo futuro puede ampliarse la biblioteca de casos, incorporar depuración paso a paso y mejorar la compatibilidad entre sistemas operativos.

25. Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2.ª ed.). Pearson.

Gerlich, M. (2025). AI Tools in Society: Impacts on Cognitive Offloading and the Future of Critical Thinking. *Societies*, *15*(1), Article 6. https://doi.org/10.3390/soc15010006

Navarro Jiménez, M., Wong Herrera, M., y Mora Urbina, A. (2026). Inteligencia Artificial: Deteriorante del Pensamiento Crítico. Del zacate al papel: hacia una visión crítica de la tecnología, (7). https://revistas.tec.ac.cr/index.php/delzacate/article/view/8243

Python Software Foundation. (s. f.). Python documentation. https://docs.python.org/3/

Sebesta, R. W. (2016). Concepts of Programming Languages (11.ª ed.). Pearson.

26. Anexos

Anexo A. Ejemplo de código Enigma

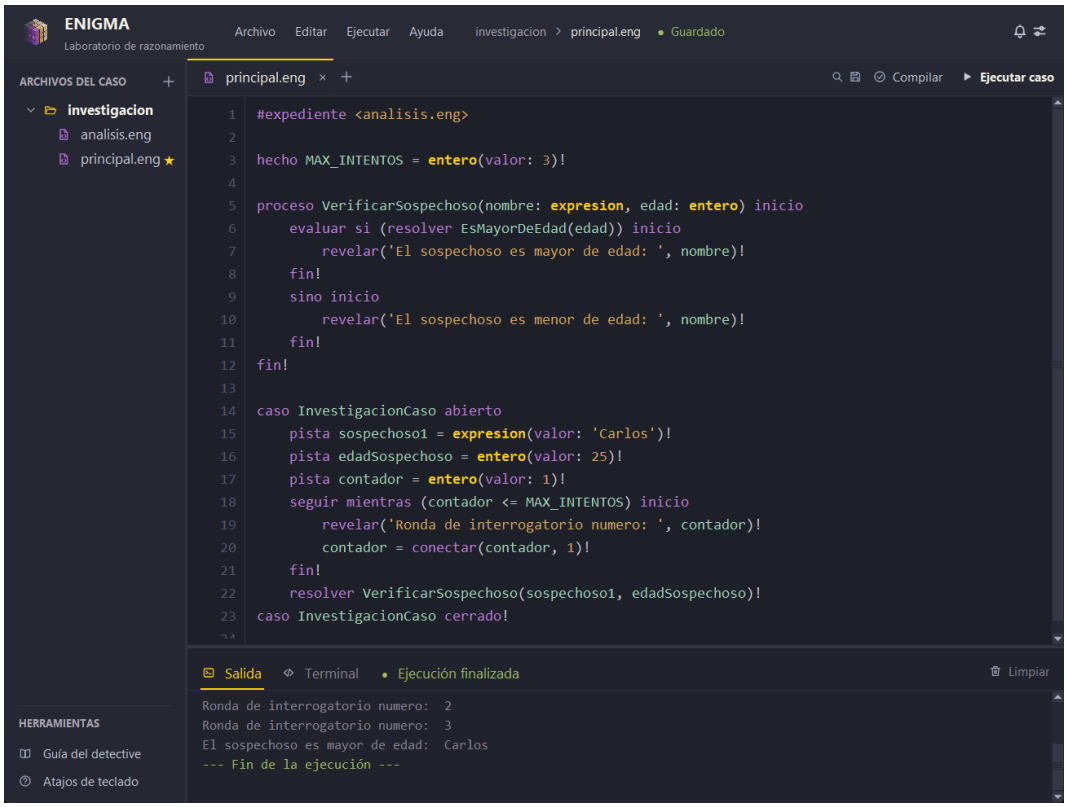


Figura A1. Programa de ejemplo InvestigacionCaso (.eng)

Anexo B. Logotipo



Figura B1. Logotipo de Enigma (un dado)

Anexo C. Ventana principal del IDE

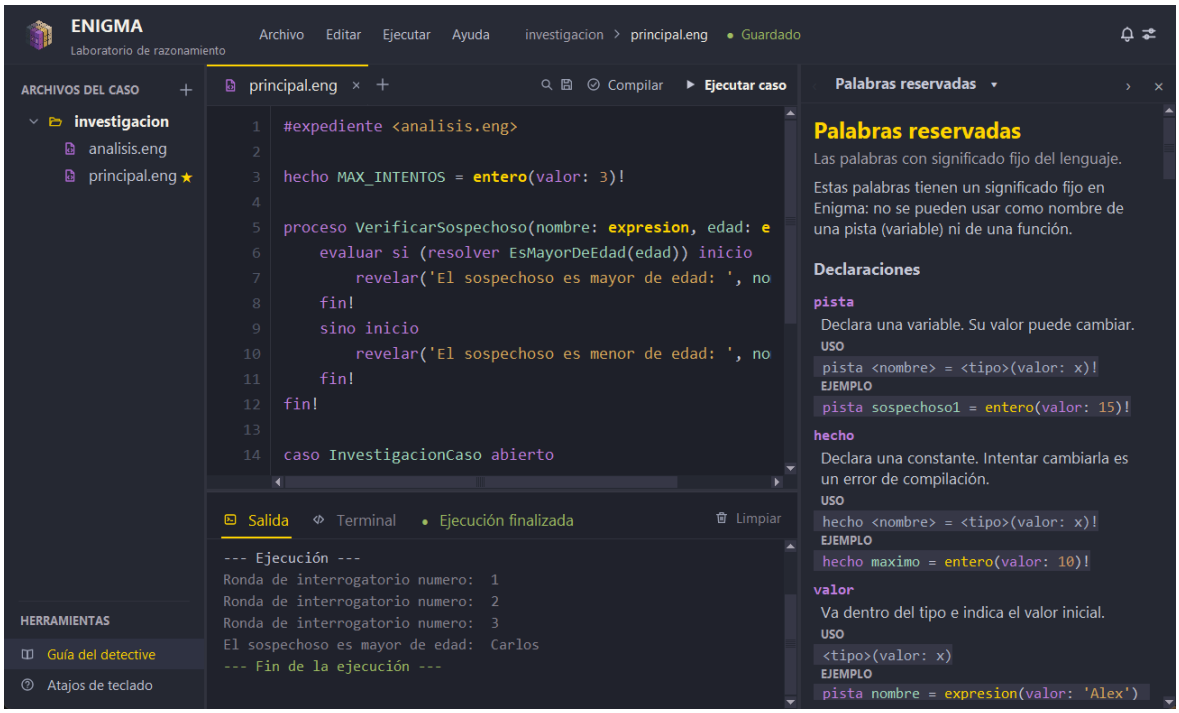


Figura C1. Ventana principal con árbol de proyectos, editor, panel de salida y guía del detective.

Anexo D. Casos de práctica

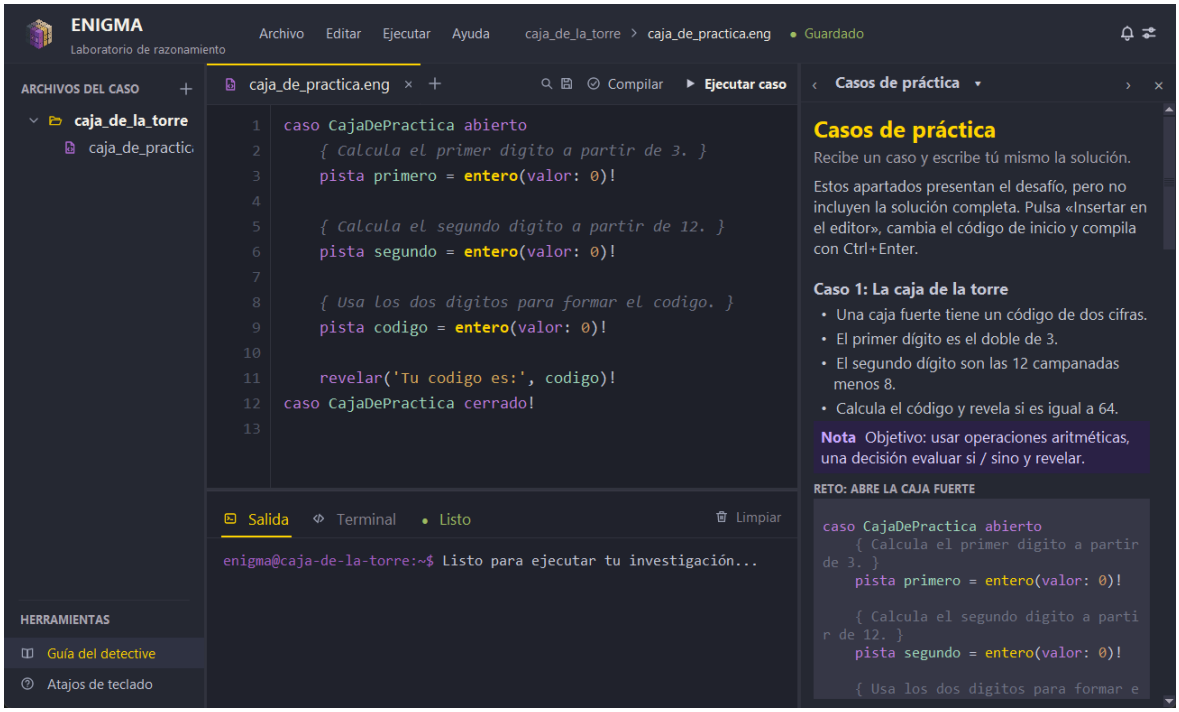


Figura D1. Ejemplo de caso de práctica abierto en el editor.

Anexo E. Retroalimentación de errores

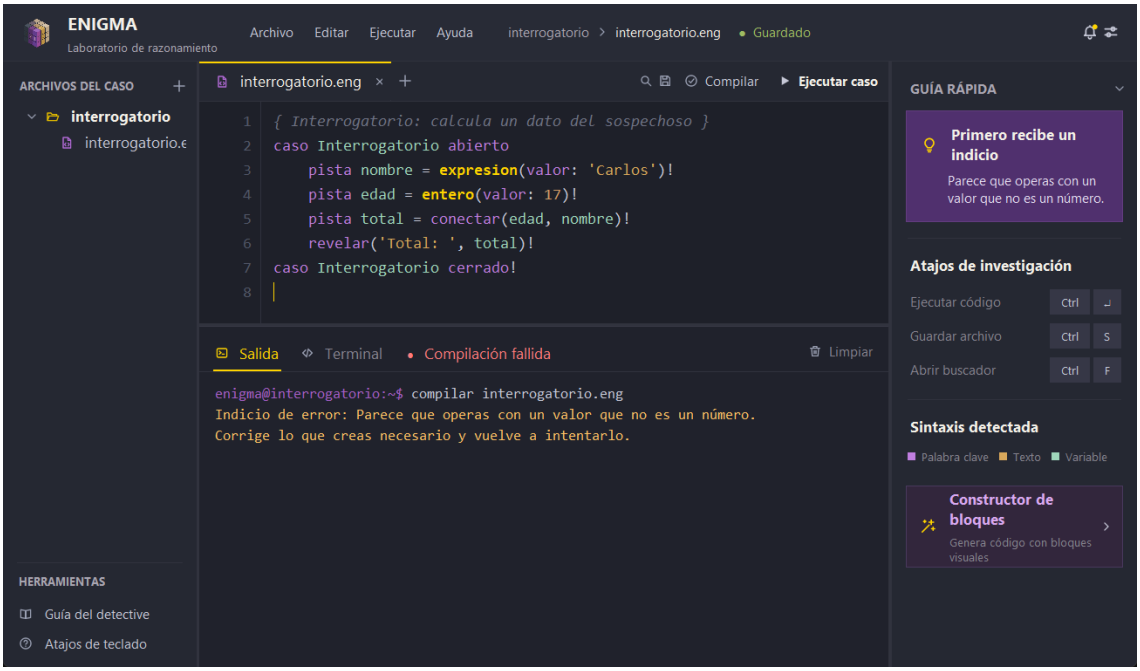


Figura E1. Primer paso: el IDE muestra solo un indicio del error.

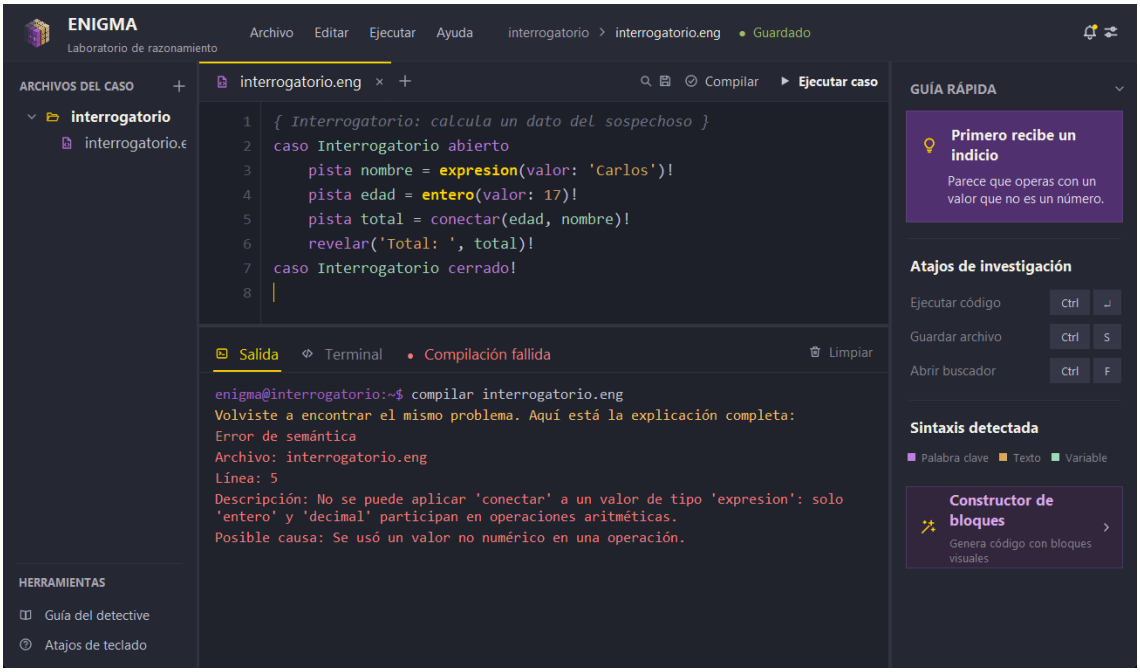


Figura E2. Segundo paso: la explicación completa del mismo error.

Anexo F. Pruebas automatizadas

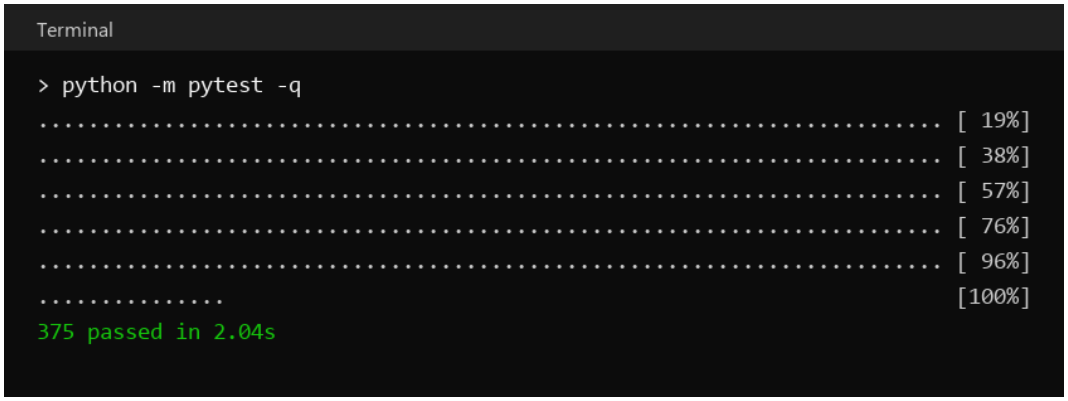


Figura F1. Resultado de la ejecución de las pruebas con pytest.